DAY 10 / 11 · WEB DEV IN 11 DAYS · GUIDED

How real apps are built

A taste of React and of servers

ShowShelf after today: **a grid built from React components, and a My List server of your own**

TODAY

The plan for the next 90 minutes

Minutes	Topic
10	Recap
12	A real React project, on my laptop
14	Components, JSX and props
12	Lists and state
14	A real server, on my laptop
13	Routes, methods and status codes
15	This afternoon in the Lab

RECAP · DAY 9

Five quick questions

1. What is a view, and how does ShowShelf pick which one shows?
2. What does ?embed[]=cast&embed[]=seasons add to /shows/169?
3. Why textContent and never innerHTML for a summary?
4. TVmaze answers 404. Is the internet broken? What do we do?
5. list.includes(169) on a list of objects gives false. Why?

ON MY LAPTOP

A real React project

```
cd day-10-demo/showshelf-react
npm install # once, before class
npm run dev # open http://localhost:5173
```

Node.js runs the tools. **npm** downloads packages. **Vite** runs the project while you work.

Real project	In the Lab
React installed with npm	already loaded
many files, with import lines	one code box: App.jsx
useState	React.useState
localhost:5173 in the browser	the preview

COMPONENTS

A function that returns what to show

The name starts with a **capital letter**. Use it like a tag: **<Hello />**.

JSX looks like HTML. No quotes around it.

{ } means: this part is JavaScript. **className**, not class.

```
function Hello() {
  return (
    <p className="meta">
      A week has {7 * 24} hours.
    </p>
  );
}

ReactDOM.createRoot(
  document.getElementById("root")
).render(<Hello />);
```

<hello /> with a small h: React looks for an HTML tag and draws nothing.

PROPS

A component's inputs

```
function Badge({ text }) {
  return <span className="badge">{text}</span>;
}
```

Like a function's parameters: one component, many results.

Text in quotes. Any other value in **{ }**.

```
<Badge text="Drama" />
<Badge text="Comedy" />
<Badge text={2 + 2} /> // shows 4

// in the real project: three props
function ShowCard({
  show,
  isInList = false,
  onToggleList,
}) { ... }
```

LISTS

map turns data into components

One component for every item in an array.

key: a name tag for each item, different for every one. An id is best.

```
const genres = [
  "Drama", "Comedy", "Fantasy",
];

function Hello() {
  return (
    <p>
      {genres.map((genre) =>
        <Badge key={genre} text={genre} />
      )}
    </p>
  );
}
```

STATE

A component's memory

useState(0) gives two things: the value, and a function that changes it.

Call the setter, and React **draws the component again**. You never touch the elements.

onClick={setWatched(watched + 1)} without () => :
“Too many re-renders”.

```
function EpisodeCounter() {
  const [watched, setWatched] =
    React.useState(0);

  return (
    <button onClick={() =>
      setWatched(watched + 1)}>
      Episodes watched: {watched}
    </button>
  );
}
```

ON MY LAPTOP

A real Express server

```
cd day-10-demo/showshelf-api
npm run dev
# ShowShelf API running on http://localhost:3000
```

localhost:3000/api/mylist answers [ ]. Add a show, ask again: it's there, saved in mylist.json.

Real server	In the Lab
Node.js runs server.js in a terminal	a pretend Express in the preview
My List in mylist.json, a file	let myList, in memory
requests.http or curl	the page and Try a request
the log in the terminal	the log in the Console

ROUTES

A method, an address and a function

req is the request that arrived. **res** is how you answer.

res.json(…) sends data back, as JSON.

A **:id** in the address arrives as **text** in req.params.

```
app.get("/api/greeting", (req, res) => {
  res.json({ text: "Good morning, class!" });
});
```

```
app.get("/api/rooms/:id", (req, res) => {
  const id = Number(req.params.id); // "2" → 2
  ...
});
```

A typo in the address: 404, Cannot GET /api/greetings.

METHODS AND STATUS CODES

Say what you want, hear what happened

Method	Means
`GET`	give me something
`POST`	save this new thing (a body: req.body)
`DELETE`	remove something

Code	Means
`200`	OK
`201`	Created
`204`	No Content: done, nothing to send
`400`	Bad Request: your data is wrong
`404`	Not Found
`500`	the server crashed

The first digit: 2 all fine · 4 the request was wrong · 5 the server has a problem

SERVERS

Never trust what arrives. Check it first, then answer with a status code.

THIS AFTERNOON IN THE LAB

Twelve steps, two parts

Steps 1–3

React: first pieces

1 Your first React component

2 JavaScript values inside JSX

3 A ShowCard component with props

Steps 4–6

React: lists and state

4 A card for every show

5 Remember something with state

6 A search box that filters

Steps 7–9

Server: first routes

7 Your first server route

8 Send My List

9 Add a show with POST

Steps 10–12

Server: rules and logs

10 Say no to bad data

11 Remove a show with DELETE

12 Read the request log

Milestone: your server adds a show, refuses an empty name, removes a show, and you can read its log.

STUCK?

In this order

1

Read the message

The “Not yet” box says what to fix.

2

Console

A red line names the problem and the line number.

3

Hint

Two per step. The second is nearly the answer.

4

Answer

Compare it with yours. Find the difference.

5

Ask

Your teacher, or the person next to you.

Empty React preview? Look at the Console first: JSX mistakes come with a line number.

END OF THE DAY

Before you go

1. What is a component, and how does it get its input?
2. What happens when you call a state setter?
3. 201, 400, 404: what do they mean?

FINISHED EARLY?

Extra challenges

Shows from TVmaze with useEffect · Ask for one show · Change a status with PATCH

TOMORROW

Make it yours and publish

Your name, your colours, your features, and your site online. Bring an email address you can open in class.